Image Generation with Vanilla GAN and DCGAN

Prof Prathosh A P

October 02, 2025

1 Introduction

This assignment extends the previous **classification** task to a **generative modeling** task on the *same dataset*. You will implement and train two generative adversarial networks: (i) a **Vanilla GAN** with fully connected Generator/Discriminator for low-resolution flattened inputs, and (ii) a **Deep Convolutional GAN (DCGAN)** with convolutional architectures suitable for image synthesis. Your goal is to learn the data distribution and generate realistic images, while documenting training stability and quantitative/qualitative performance.

2 Dataset & Preprocessing

Use exactly the **same dataset, splits, and preprocessing** pipeline as in the classification program file (dataset). In particular:

- Keep the same train/val/test split definitions.
- Apply the same normalization statistics (mean/std) to real images fed to the Discriminator; ensure that the Generator output is mapped back to the same scale for visualization.
- If images were resized/cropped earlier, use the same spatial resolution here.

3 Task Objectives

1. **Implement Vanilla GAN** (Goodfellow et al., 2014): a fully connected Generator $G(z)$ and Discriminator $D(x)$ trained with the standard minimax (or non-saturating) losses.
2. **Implement DCGAN** (Radford et al., 2015): convolutional Generator/Discriminator with strided (de)convolutions, batch norm (except in D's output), and `ReLU`/`LeakyReLU` activations.
3. **Train both models** on the training split; use the validation split to tune hyperparameters and **early stopping** (by monitoring FID or a proxy metric).
4. **Evaluate** on the test split with quantitative metrics (e.g., FID/KID, Inception Score where meaningful) and **qualitative grids** of generated samples.
5. **Report** stability tips (learning rates, βs for Adam, label smoothing, one-sided label smoothing, input noise, spectral norm or weight clipping if tried) and show learning curves.

4 Recommended Architectures

4.1 Vanilla GAN (fully connected)

Use a latent vector $z \sim \mathcal{N}(0, I)$ (dimension 64–128). Flatten images to vectors of size $d = H \times W \times C$.

- **Generator** G: `Linear` $\rightarrow$ `BatchNorm1d` $\rightarrow$ `ReLU` (repeat 2–3 blocks), final `Linear` to d, output activation `Tanh` (if using normalized images in $[-1, 1]$) or `Sigmoid` (for $[0, 1]$).
- **Discriminator** D: MLP with `LeakyReLU(0.2)`, optional `Dropout`, final `Linear` to 1 with `BCEWithLogitsLoss`.

4.2 DCGAN (convolutional)

Adopt DCGAN guidelines: no pooling, use strided convs; `BatchNorm2d` in G (and often in early D layers), `ReLU` in G, `LeakyReLU(0.2)` in D; `Tanh` output for G.

- **Generator** G: `ConvTranspose2d` blocks doubling spatial dims each step until $H \times W$, channels descending to C, final `Tanh`.
- **Discriminator** D: `Conv2d` blocks halving spatial dims, increasing channels, final `Linear` to 1 (via global flatten), `BCEWithLogitsLoss`.

5 Losses & Optimization

- Use either the **minimax loss** or the **non-saturating loss** ($\max_D \ \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$; $\min_G \ -\mathbb{E}[\log D(G(z))]$).
- Optimizer: **Adam** with typical DCGAN settings (α=2×10^{-4}, β_1=0.5, β_2=0.999). Try 1:1 update ratio of $D : G$ per step.
- Stabilization options to try and document: label smoothing, instance noise, spectral norm (on D), gradient penalty (optional), EMA of G weights (optional).

6 Self Evaluation Protocol

6.1 Quantitative

Compute at least one of the following on the **test** split (or a held-out set):

- **FID** (Fréchet Inception Distance) — lower is better.
- **KID** (Kernel Inception Distance) — lower is better.
- **Inception Score** (where applicable) — higher is better.

If computing FID/KID is infeasible, report proxy metrics: **nearest-neighbor L2 in feature space**, **precision/recall for generative models** (if available), or classifier-based realism (passing generated images through your classifier to estimate class confidence).

6.2 Qualitative

- Grids of 8×8 (or similar) generated samples at multiple training epochs (e.g., epochs 1, 10, 25, 50, final).
- **Interpolation** in latent space: linearly interpolate between two z's and visualize morphing.
- **Mode coverage** check: per-class samples if you attempt a class-conditional variant (optional).

7 Experiments

1. **Baseline Vanilla GAN** vs **baseline DCGAN**: compare training stability, sample quality, and metrics.
2. Sensitivity to **latent dimension** (z dim), **batch size**, and **learning rate**.
3. Effect of **stabilization tricks**: label smoothing, spectral norm, or instance noise.
4. (Optional) **Class-conditional** DCGAN using a projection discriminator or conditional batch norm.

8 References

- I. Goodfellow et al., "Generative Adversarial Nets," NeurIPS 2014.
- A. Radford, L. Metz, S. Chintala, "Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks," ICLR 2016.
- M. Heusel et al., "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium" (FID), NeurIPS 2017.